

Interaction Portal (v2)

What is the Interaction Portal? (Overview for Human Users)

The **Interaction Portal** is an interactive, browser-based collaborative workspace where human participants work together with a team of artificial intelligence (AI) agents to solve realistic business and marketing strategy challenges. In this study, you step into the role of a team lead or strategic decision-maker addressing client briefs--such as launching an educational campaign, redesigning a brand, or planning an event.

How Users Use It:

- **Meet the AI Team:** You collaborate with four specialized AI domain experts--**Nia** (Creative Concept & Copywriting), **Theo** (Audience & Market Research), **Rhys** (Channels & Content Distribution), and **Mira** (Budget, Feasibility & Metrics)--coordinated by an **AI Orchestrator** that synthesizes team progress.
- **Interactive Live Chat:** You communicate with the team via a familiar group chat interface, directing questions to specific specialists using **@Nia**, **@Theo**, **@Rhys**, **@Mira**, or broadcasting questions to **@team**.
- **Real-Time Guidance & Feedback Panel:** As you and the agents brainstorm, an intelligent side panel continuously summarizes team progress, diagnoses what requirements might be missing or unverified, and provides strategic coaching.
- **Creating the Deliverable:** Depending on your experimental condition, you either observe the AI drafting the recommendation card (Passive), review and approve/reject/modify proposed card points using Keep, Cut, and Send-back controls (Evaluative), or write the strategic card yourself with AI-suggested inspirations (Generative). In the baseline condition (No-AI), you solve the challenges independently using a structured digital Reference Pack containing all source materials.
- **Reflections & Breaks:** After submitting your strategic recommendation for each client, you complete short reflection check-ins. A structured break occurs after the third client to ensure you stay refreshed.

Why This Study Matters (The Research Objective): As generative AI transforms modern workplaces, teams increasingly involve humans working alongside multiple autonomous AI agents. This research portal investigates **how different levels of human control (agency)** and **different styles of AI management feedback** impact problem-solving quality, critical thinking, creative output, and human satisfaction. By tracking keystrokes, messages, review decisions, and time spent on reference materials in a secure, privacy-preserving event log, this platform provides scientific insights into how future AI systems can be designed to empower human workers rather than overwhelm or replace them.

1. Research Design & Conditions

The experiment employs a 3 x 3 + 1 between-subjects factorial design (10 conditions total):

- **Participant Role** (3 levels):
 - **Passive (PAS):** Read-only deliverable card; participant observes team drafting and submits.
 - **Evaluative (EVA):** Participant acts as reviewer using Keep, Cut, and Send-back controls on AI proposals.
 - **Generative (GEN):** Participant writes card content directly, supported by AI suggestions (must exceed a minimum typed share, default 25%).
- **Orchestrator Panel Feedback** (3 levels):
 - **Coordination (COORD):** Status reports on specialist contributions and task assignments.
 - **Task-Focused (TASK):** Status reports plus diagnosis of what is missing or unverified on the deliverable card.
 - **Developmental (DEV):** Status, card diagnosis, plus recurring conceptual principles explaining why errors arise.
- **No-AI Baseline (NOAI):** - No chat, no agents, no panel. The participant works through the client tasks using a four-part collapsible **Reference Pack** containing the exact source materials held across specialists.

Condition Matrix

Code	Role	Panel Level	Team Chat	Card Editor Mode
PAS-COORD	Passive	Coordination	Active	Read-only
PAS-TASK	Passive	Task-Focused	Active	Read-only
PAS-DEV	Passive	Developmental	Active	Read-only
EVA-COORD	Evaluative	Coordination	Active	Keep / Cut / Send-back
EVA-TASK	Evaluative	Task-Focused	Active	Keep / Cut / Send-back
EVA-DEV	Evaluative	Developmental	Active	Keep / Cut / Send-back
GEN-COORD	Generative	Coordination	Active	Typing + AI suggestions (min 25% typed)
GEN-TASK	Generative	Task-Focused	Active	Typing + AI suggestions (min 25% typed)
GEN-DEV	Generative	Developmental	Active	Typing + AI suggestions (min 25% typed)

Code	Role	Panel Level	Team Chat	Card Editor Mode
NOAI	None	None	Disabled	Reference pack + manual authoring

2. Architecture & Folder Structure

```
interaction_portal/
|-- app/                # Application runtime and backend mechanics
|   |-- agents/         # LLM adapters, structured schemas, specialist & orchestrator logic
|   |-- harnesses/      # Automated verification harnesses (Harness A & Harness B)
|   |-- simulation/     # State coordination, card provenance, chat routing, practice
|   |-- static/         # Client-side CSS styling and vanilla JS (telemetry, editor, chat)
|   |-- templates/      # Jinja2 HTML templates for each experimental screen
|   |-- config_loader.py # Configuration parser and validator
|   |-- event_store.py  # Append-only SQLite telemetry database engine
|   |-- main.py         # FastAPI web application factory & routes
|   \-- state_machine.py # Forward-only page flow engine & authoritative server timers
-- config/             # Experiment content, prompts, condition matrix, UI text, and parameters
|   |-- conditions.yaml # 10 condition definitions (role x panel level)
|   |-- config.yaml     # Master configuration file (UI text, timers, card fields, practice)
|   |-- content/clients/ # Client folders (C1..C7) with public briefs and private materials
|   \-- prompts/        # System prompts, orchestrator planning prompts, panel level templates
-- logs/               # Telemetry database storage (portal.sqlite3)
-- tests/              # Automated pytest unit and integration test suite
-- md_to_pdf.py        # Documentation export utility (converts README.md to PDF)
-- requirements.txt     # Python dependencies
\-- README.md          # This documentation file
```

3. Detailed Guide to Each Folder

3.1 app/ -- Application Engine (For the Application Hoster)

Intention & Purpose: The `app/` directory contains the core FastAPI web application and simulation runtime. It manages client sessions, coordinates team chat streaming via NDJSON, enforces authoritative wall-clock server timers, tracks span-by-span deliverable card provenance, and logs fine-grained telemetry to SQLite.

Why Minimal Tweaking is Needed: The code in `app/` is designed as a generic, headless experimental motor. All experiment-specific wording, prompts, client briefs, agent personalities, and timing parameters are completely decoupled into `config/`. As an application hoster or sysadmin, you do not need to modify Python code in `app/` under normal operation. You only need to configure standard deployment environment variables (host, port, API keys, secret cookies) and run the service.

Core Submodules: - `app/agents/`: LLM client interfaces (`OpenAICompatClient` and deterministic `MockClient`), Pydantic output schemas, keyword domain classifiers (`topics.py`), and orchestrator panel engines. - `app/simulation/`: Multi-client session manager (`session_manager.py`), card span provenance and role authoring rules (`card_provenance.py`), `@mention` chat routing and folded internal block grouper (`chat_router.py`), and scripted onboarding practice (`practice.py`). - `app/event_store.py`: Append-only SQLite data store equipped with SQL triggers that abort any `UPDATE` or `DELETE` statements on events. - `app/state_machine.py`: Deterministic forward-only state progression engine ensuring participants cannot jump backwards or reset timers via browser reloads. - `app/main.py`: FastAPI application factory, signed session cookie management, streaming endpoints, and token-authenticated data export handlers. - `app/static/` & `app/templates/`: Dependency-free vanilla JavaScript (attention telemetry, card editor, auto-expanding chat) and clean semantic Jinja2 HTML templates.

3.2 config/ -- Study Configuration & Customization (For the Researcher / Coder)

Intention & Purpose: The `config/` directory is the single source of truth for the entire study. Everything participant-facing or model-facing lives here. By modifying these YAML and Markdown files, researchers can update screen text, rewrite prompts, adjust timers, or add entirely new client tasks without touching the Python backend.

How to Update and Tweak Configurations:

- Screen Text and Wording (`config/config.yaml` to `pages`):** - All text displayed to participants across all 11 screens is defined under the `pages` dictionary in `config/config.yaml`. - **Common captions:** `pages.common.continue`, `pages.common.submit`, `pages.common.unlocks_in`. - **Screen-specific instructions:**

- **pages.orientation**: Orientation notes, privacy policy text, and requirement lists.
 - **pages.team**: Descriptions of specialists and the orchestrator card.
 - **pages.role**: Explanations for Passive, Evaluative, and Generative roles.
 - **pages.task**: Chat placeholders, unknown mention error templates, and card status notices.
 - **pages.checkin**: Post-task survey questions (**ci1..ci5**), Likert scale ranges, and multiple-choice options.
 - **pages.break**: Break instructions and early-finish button text.
 - **pages.exit**: Completion thank-you message and Qualtrics return instructions.
2. **Agent Behavior & System Prompts (**config/prompts/**):** - **Specialists (**config/prompts/specialists_system.md**):** System prompt defining the specialist's role, behavioral boundaries, JSON response schema, refusal rules for out-of-domain questions, and prohibitions on unprompted disclosure. - **Orchestrator Planning (**config/prompts/orchestrator_plan.md**):** Instructions guiding how the Orchestrator assigns tasks to specialists during opening rounds and coordination bursts. - **Orchestrator Panels (**config/prompts/panel_levels/**):**
- **coordination.md**: Prompts the model to report specialist activity and contributions.
 - **task_focused.md**: Instructs the model to diagnose what is missing or unverified on the deliverable card against brief requirements.
 - **developmental.md**: Instructs the model to explain the recurring conceptual rationale for common errors.
 - **_common.md**: Shared panel constraints, including the strict 60-80 word budget, forbidden imperative language, and required section headers.
 - **panel_compress.md**: Fallback compression prompt triggered if initial panel output exceeds 80 words.
3. **Client Tasks & Hidden Materials (**config/content/clients/**):** - Seven client tasks (**C1** through **C7**) are provided by default. Each folder contains:
- **brief.md**: Public client brief with frontmatter **requirements: [...]** that participants read.
 - **private/nia.md, private/theo.md, private/rhys.md, private/mira.md**: Private materials held exclusively by each specialist.
 - **Hidden Items**: Marked in private files using HTML comments: **markdown <!-- hidden:C1-H1 --> The client privately stated that they will not accept any video shorter than 30 seconds. <!-- /hidden:C1-H1 -->** These markers allow the system to automatically track whether a specialist has disclosed a hidden constraint.
 - **Adding a New Client**: Create **config/content/clients/C8/** with **brief.md** and the 4 **private/*.md** files, then add **C8** to **clients.randomised** in **config/config.yaml**.
4. **Timers & Pacing (**config/config.yaml** to **timers:**):** - **timers.task_seconds**: Duration of each client task (default **600** = 10 minutes). - **timers.break_seconds**: Mid-session break duration after client 3 (default **120** = 2 minutes). - **timers.orientation_min_seconds**: Minimum reading lockout before "Continue" unlocks (default **15**). - **timers.resume_gap_seconds**: Participant inactivity cutoff (default **900** = 15 minutes). After this window, re-entry is blocked as **abandoned_timeout**. - **testing.timer_scale**: Global speed multiplier for testing (e.g. set to **10.0** or **100.0** to shrink timers for rapid walkthroughs).
5. **Conditions Matrix (**config/conditions.yaml**):** - Maps condition codes (**PAS-COORD, EVA-TASK, GEN-DEV, NOAI**, etc.) to their active parameters (**role, panel_level, ai**).

3.3 logs/ -- Telemetry & Database Storage (How to Open and Read Logs)

Intention & Purpose: The **logs/** folder stores the append-only SQLite database (**logs/portal.sqlite3**). This database logs every participant action, agent reply, keystroke interaction, scroll sample, window blur/focus event, and card submission with millisecond UTC timestamps.

Database Tables Overview:

- **events**: Core telemetry stream (**pid, timestamp, client_id, page_id, event_type, autopilot, metadata** JSON). SQL triggers prevent any modification or deletion.
- **sessions**: Participant session metadata (**pid, condition_code, started_at, ended_at, end_state, resume_count, completion_code, autopilot, state** JSON).
- **transcripts**: Full chronological chat transcripts per participant and client task, including private folded blocks.
- **panels**: Snapshots of every Orchestrator side-panel generated (**level, text, word_count, sections_present**).
- **cards**: Final and intermediate deliverable cards with span-by-span author attribution and history.

How to Open and Inspect Logs:

1. **Option A: DB Browser for SQLite (Recommended GUI)** - Download and open [DB Browser for SQLite](#). - Open [interaction_portal/logs/portal.sqlite3](#). - Navigate to the **Browse Data** tab to view tables, or use **Execute SQL** to run queries.
2. **Option B: Command Line (sqlite3)** `bash sqlite3 logs/portal.sqlite3` Useful SQL queries: ````sql -- View recent participant events SELECT timestamp, pid, page_id, event_type, metadata FROM events ORDER BY id DESC LIMIT 20;`
`-- Check submitted cards with typed share percentage SELECT pid, client_id, timestamp, json_extract(card, '.typed_share') AS typed_share FROM cards;`
`-- Check post-task check-in responses SELECT timestamp, pid, client_id, json_extract(metadata, '.item_id') AS item, json_extract(metadata, '.value') AS answer FROM events WHERE event_type = 'checkin_answer'; ````
3. **Option C: Direct Web Data Exports (No SQL Needed)** While the server is running, download structured data directly via protected admin endpoints using **PORTAL_ADMIN_TOKEN**:

In Windows PowerShell: ````powershell # Export all events as CSV curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/events.csv -o events.csv`

`# Export all submitted cards as JSON curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/cards.json -o cards.json`

`# Export complete database dump as JSON curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/all.json -o all_data.json`

`# Or using PowerShell's native Invoke-WebRequest: Invoke-WebRequest -Uri "http://localhost:8000/admin/export/events.csv?token=admin-test-token" -OutFile events.csv ````

In Windows Command Prompt (cmd.exe): `cmd curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/events.csv -o events.csv curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/cards.json -o cards.json curl.exe -H "x-admin-token: admin-test-token" http://localhost:8000/admin/export/all.json -o all_data.json`

Direct Browser Download (Easiest on Windows): Simply open any browser and navigate to: -

<http://localhost:8000/admin/export/events.csv?token=admin-test-token> -

<http://localhost:8000/admin/export/cards.json?token=admin-test-token> -

<http://localhost:8000/admin/export/all.json?token=admin-test-token>

3.4 app/harnesses/ -- Automated Verification Test Harnesses

Intention & Purpose: These harnesses provide automated verification of information boundaries and simulation stability **before** running human participants. They are rerun whenever model versions or system prompts are modified.

1. Harness A: Behavioural Rules & Boundary Verification ([app/harnesses/harness_a.py](#))

Executes an automated battery of adversarial probes against specialists and the Orchestrator to verify 4 behavioral rules: - **Rule 1 (No Unstated Facts):** Specialists never invent client facts not present in their private materials. - **Rule 2 (No Unasked Disclosure):** Specialists do NOT disclose hidden items during general questions or broadcasts; disclosure only occurs when explicitly asked about that specific topic. - **Rule 3 (No Domain Crossover):** Specialists decline and redirect questions outside their designated domain (e.g. Nia asked about budget declines and redirects to Mira). - **Rule 4 (No Orchestrator Hallucination):** The Orchestrator asserts no client facts, holds no private material, and assigns work based solely on the public brief.

How to run:

```
# Test single client (C1)
python -m app.harnesses.harness_a --config config/config.yaml --client C1

# Test all clients (C1 through C7)
python -m app.harnesses.harness_a --client all
```

Outputs a structured test report ([harness_a_report.json](#)) with pass/fail status per probe.

2. Harness B: Autopilot End-to-End Simulation ([app/harnesses/harness_b.py](#))

Replaces the human participant with an automated scripted driver to execute complete sessions end-to-end: - Exercises orientation, team, role, and scripted practice onboarding (including role checks). - Triggers opening bursts, verifies folded block generation (`blk_*`), and checks reply latencies. - Sends broadcast queries and direct `@mentions` to all specialists. - Validates that panel generation adheres strictly to the 60-80 word budget and condition sections. - Exercises card editor actions per role (passive verification, evaluative keep/cut/send-back, generative typing share). - Submits cards, answers check-ins, and verifies that all emitted telemetry events are tagged with `"autopilot": true`.

How to run:

```
# Test single condition and client
python -m app.harnesses.harness_b --condition EVA-TASK --client C2

# Test all 10 experimental conditions
python -m app.harnesses.harness_b --all-conditions

# Test complete 7-client session to exit screen
python -m app.harnesses.harness_b --condition GEN-DEV --full-session
```

Outputs a structured execution report (*harness_b_report.json*).

3.5 tests/ -- Automated Test Suite

Intention & Purpose: The `tests/` directory contains a 37-test automated test suite using `pytest`. It verifies backend mechanics, security guarantees, data integrity, and API routes.

Test Modules:

- `test_config_loader.py`: Validates YAML parsing, condition mappings, private files, and timer scaling.
- `test_event_store.py`: Tests append-only SQL triggers (rejecting UPDATE/DELETE), atomic single-use session locks, and CSV/JSON export formatting.
- `test_state_machine.py`: Tests forward-only page flows, NOAI page skips, break triggers, 7th client alone mode, and wall-clock deadlines.
- `test_card_provenance.py`: Validates span tracking, keep/cut/send-back actions, and generative typed share enforcement.
- `test_chat_router.py`: Tests `@mention` parsing, broadcast aliases, fuzzy typo suggestions, and folded block grouping.
- `test_agents_and_mock.py`: Verifies specialist structured outputs, orchestrator burst planning, and panel word budget guardrails.
- `test_practice.py`: Tests scripted onboarding exercises and wrong role-check redirect penalties.
- `test_session_manager.py`: Tests session resume rules, 15-minute gap timeouts, chat streaming, and check-in validation.
- `test_api_endpoints.py`: Tests FastAPI routes, signed session cookies, desktop vs mobile filtering, and admin token authorization.
- `test_harnesses.py`: Programmatic execution of Harness A and Harness B.
- `test_terminal.py`: Validates ANSI stripping fallback, Windows VT console color detection, and colorize helpers.

How to run tests:

```
# Run all tests (46 total)
pytest tests -v

# Run a specific test file
pytest tests/test_event_store.py -v

# Run with stdout output enabled
pytest tests -s
```

3.6 requirements.txt -- Python Dependencies Breakdown

Intention & Purpose: `requirements.txt` lists all external Python packages required to run and test the portal.

Package	Purpose
fastapi	High-performance ASGI web framework providing HTTP routes and NDJSON chat streaming.
uvicorn	Production-ready ASGI server for serving the FastAPI application.
starlette	Core ASGI toolkit underlying FastAPI (used for request handling and test client).
jinja2	Template engine for rendering participant-facing HTML pages.
python-multipart	Parser for handling HTML form submissions.

Package	Purpose
<code>pydantic</code>	Data validation and schema enforcement for structured LLM outputs.
<code>pyyaml</code>	Parser for loading <code>config.yaml</code> , <code>conditions.yaml</code> , and Markdown frontmatter.
<code>itsdangerous</code>	Cryptographic URL-safe serializer used for tamper-proof session cookies.
<code>httpx</code>	Async HTTP client used by the test client and external web requests.
<code>openai</code>	Official client library for talking to OpenAI-compatible LLM endpoints.
<code>pytest</code>	Test execution framework for running the automated unit and integration suite.
<code>colorama</code>	Cross-platform colored terminal text support on Windows (Anaconda Prompt, <code>cmd.exe</code>).
<code>markdown</code>	Markdown parsing and HTML compilation for documentation export.
<code>xhtml2pdf</code>	HTML/CSS to PDF rendering engine for generating standalone PDF documentation.

Tip

****Anaconda Prompt & Windows Terminal Colors:**** The application automatically initializes Windows console color handling at startup (`app.terminal.init_terminal()`). In Anaconda Prompt and `cmd.exe`, it activates Windows Virtual Terminal (VT) processing and `colorama` so that Uvicorn server logs and harness reports display in vibrant colors without raw escape artifacts (such as `<[32m`, `<[0m`, `<[36m`, `<[1m`). If colors are not supported (e.g. redirected log files or non-VT terminals), a built-in fallback cleanly omits and strips all ANSI escape codes.

Installation:

```
pip install -r requirements.txt
```

3.7 `md_to_pdf.py` -- Markdown-to-PDF Documentation Exporter

Intention & Purpose: `md_to_pdf.py` converts Markdown documentation files into styled, publication-ready A4 PDF documents. It formats headings, tables, callout banners, and code blocks with running headers, footers, and page numbers.

Usage:

```
# Convert README.md to README.pdf (default)
python md_to_pdf.py

# Convert any custom Markdown file
python md_to_pdf.py docs/guide.md docs/guide.pdf
```

4. Running the Application on Windows

4.1 Standalone Test Mode (No Qualtrics, Offline Mock)

Ideal for local testing, rapid demonstration, and manual review without network access or API keys.

1. **Activate your virtual environment (if using one):** - PowerShell: `.\.venv\Scripts\Activate.ps1` - Command Prompt: `.\.venv\Scripts\activate.bat`

2. **Launch the application:**

In Windows PowerShell: `powershell env:PORTAL_STANDALONE="1" python -m app.main`

In Windows Command Prompt (cmd.exe): `cmd set PORTAL_STANDALONE=1 python -m app.main`

3. **Access the portal:** Open your browser to <http://localhost:8000/>. A test launcher allows you to select any of the 10 experimental conditions with an automatically generated participant ID (`pid`). The completion code is displayed on the exit page without redirecting to Qualtrics.

4.2 Production Deployment on Windows (Qualtrics & Real Model)

In production, participants arrive via Qualtrics with their assigned **pid** and condition code in the query parameters:

```
http://<host>:8000/?pid=R_123456789&condition=EVA-TASK
```

Before launching the server, set the production environment variables in your Windows terminal:

In Windows PowerShell:

```
# Disable standalone mode (requires pid and condition query parameters)
env:PORTAL_STANDALONE="0"

# Configure LLM provider and OpenAI API key
env:PORTAL_LLM_PROVIDER="openai"
env:OPENAI_API_KEY="sk-..."

# Configure secure session cookie signing key and data export token
env:PORTAL_SECRET_KEY="replace-with-a-secure-random-secret"
env:PORTAL_ADMIN_TOKEN="secure-token-for-exporting-data"

# Optional: Host and port (default is 127.0.0.1:8000)
# env:PORTAL_HOST="0.0.0.0"
# $env:PORTAL_PORT="8000"

python -m app.main
```

In Windows Command Prompt (cmd.exe):

```
:: Disable standalone mode (requires pid and condition query parameters)
set PORTAL_STANDALONE=0

:: Configure LLM provider and OpenAI API key
set PORTAL_LLM_PROVIDER=openai
set OPENAI_API_KEY=sk-...

:: Configure secure session cookie signing key and data export token
set PORTAL_SECRET_KEY=replace-with-a-secure-random-secret
set PORTAL_ADMIN_TOKEN=secure-token-for-exporting-data

python -m app.main
```

At the end of the session, the portal automatically redirects participants back to the Qualtrics post-survey with their signed completion code:

```
https://example.qualtrics.com/jfe/form/SV_demo?pid=R_123456789&code=IP2-A1B2C3D4
```